Topological Feature Learning for Parameter Estimation in Neyman–Scott Processes *

Flavio Gualtieri

August 17, 2026

Abstract

The parameters of spatial point processes are interpretable scientific quantities, but the methods available to estimate them rely on closed-form expressions for hand-picked summary statistics such as the K and g functions. This reliance limits them: a closed form must be re-derived for each new family of processes, and the choice of statistic discards information about the point pattern before any estimation takes place. We propose a deep learning estimator that instead uses vectorized persistence summaries as features, which require no closed form and no choice of summary statistic. We pursue two objectives. Firstly, we compare several topological summaries against the estimation task directly, so that their relative efficacies indicate which structure each one retains. Secondly, we use the strongest summary to build an estimator for the Thomas, Matérn cluster, and nested Thomas processes under a common reparameterization, and evaluate it against classical baselines on shared test data. Our contributions include a per-diagram calibration scheme for vectorizing across a heterogeneous population of persistence diagrams, and a simulator validated against an exact finite-window second-order identity. [TODO: headline result: [ParamNet attains a mean loss of X, against Y for minimum contrast, and degrades more gracefully as the process approaches complete spatial randomness]]

1 Introduction

The study of spatial point processes finds applications in astronomy, cell biology, epidemiology, etc. Many natural phenomena - such as galaxy clustering - are modelled using SPPs, so one can hope to leverage an understanding of these processes to obtain novel insights. [TODO: pick ONE application and carry it through the whole paper as the running example – galaxy clustering is the natural choice given the related work]

The ability to infer the parameters of SPPs is of particular interest because a wealth of canonical summary statistics become readily available once these parameters are known, such as ... [TODO: this motivation is CIRCULAR and must be replaced. Classical methods estimate parameters FROM summary statistics, not the other way round. The real argument: the parameters are the physically interpretable quantities (κ = cluster density, μ = occupancy, σ = cluster scale), and existing estimators are fragile in nameable regimes.] [FIX: The parameters of spatial point processes are of particular interest because they are interpretable scientific quantities. For example ...]

*[TODO: title is a placeholder – it should name the topological angle and the family class, not just "deep learning"]

A number of parameter estimation methods have been proposed, typically relying on closed-form expressions for summary statistics, for example the K and g functions. [TODO: these three become a short paragraph with citations, not a bullet list]

- Minimum contrast on K or g (Diggle 2003).
- Palm likelihood (Tanaka, Ogata & Stoyan 2008).
- Bayesian and MCMC.

These methods have limitations we hope to address. Firstly, existing methods rely on the existence of closed-form expressions of summary statistics – such as the K-function – which may not exist for every process. Secondly, classical parameter estimation methods suffer from hyper-parameter sensitivity, specifically the minimum contrast method depends on (r_{max}, q, p). Thirdly, classical methods become unstable as the process approaches complete spatial randomness. As the overlap index ν grows the clusters merge, so κ and μ cease to be separately identifiable from a single realization: a small parent intensity with large clusters and a large parent intensity with small ones produce point patterns whose second-order behaviour is near-indistinguishable. Minimum contrast is fitted by numerical optimization of a discrepancy that is close to flat along this trade-off, so the estimates it returns in this regime are sensitive to the starting point and to the choice of r_{max}. We deliberately retain this regime in our design distribution (see Section 5) rather than restricting to well-separated clusters, and report error as a surface over the design grid so that the degradation is visible rather than averaged away.

These limitations motivate the use of persistence-based features which provide a multi-scale, process-agnostic descriptor of the point cloud that does not require a closed-form or a choice of summary statistic. [TODO: More mechanism]

[TODO: MISSING CONTRIBUTIONS LIST. Draft it as: 1. a systematic comparison of vectorized topological summaries as features for SPP parameter estimation (the ORIGINAL objective – re-elevate it); 2. a per-diagram coverage-quantile calibration scheme for vectorizing across a heterogeneous population of diagrams, with an explicit leakage protocol; 3. a validated multi-family simulator + evaluation protocol, including an exact finite-window variance identity; 4. an estimator covering Thomas / Matérn cluster / nested Thomas under a common reparameterization. NOTE: architectural novelty is NOT on this list – see Section 2.]

[TODO: roadmap paragraph]

2 Related Work

[TODO: WRITE THIS SECTION. Concede the architecture to [A]; concede topological features for parameter estimation to [B]; claim the comparative study, the calibration methodology, and the multi-family scope.]

[TODO: one paragraph: neural estimators for SPPs – [A] and its lineage] [FIX: REF first proposed the use of deep learning for parameter estimation. They implement an encoder/inference architecture similar to PointNet, but use the L-function as the learned feature and therefore suffer from the same limirations as classical methods (i.e. reliance on a closed-form summary statistic). REF employ a similar architecture as REF, but actually use topological features. Their model is deigned to estimate cosmological parameters rather than the parameters of the underlying point process. Furhtermore, they do not systematically compare various topological summaries, nor do they address the calibration of the summaries they use.]

[TODO: one paragraph: topological summaries as features for inference – [B], plus the vectorization literature (persistence images, Betti curves)] [FIX: The vectorization of persistence diagrams is an active area of research arising from the issue that persistence diagrams – not being a

Hilbert space – cannot be direclty used as inputs for machine learning models. To mitigate this, REF propose persistence images, which vectorise persistence diagrams by applying a Gaussian kernel to each point in the diagram and intergating over a grid. REF propose Betti curves, which vectorize persistence diagrams by counting the number of topological features that are alive at each scale.]

[TODO: one paragraph: the gap, stated in one or two sentences]

3 Background

We provide the background in SPPs and persistent homology required for this discussion. Readers with a working understanding of both may skip to Section 6.

3.1 Spatial point processes

Definition 3.1: Intensity function A d-dimensional SPP X on a subset $W \subseteq \mathbb{R}^d$ is equipped with an intensity function $\rho(u) : \mathbb{R}^d \to \mathbb{R}^+$ such that

$$\mathbb{E}[N(W)] = \int_W \rho(u)du,$$

where $N(W) = \mid W \cap X \mid$ denotes the number of points of X in W. When $\rho(u) = \lambda \in \mathbb{R}$, X is said to be *homogeneous*.

Example For a homogeneous Poisson process with intensity λ, we have $\rho(u) = \frac{1}{\lambda}$, i.e. points are scattered uniformly on the space.

Definition 3.2: Neyman-Scott process Let P be a homogeneous Poisson *parent* process with intensity κ. For a Neyman-Scott process with Poisson-distributed offspring we have:

$$\rho_{NS}(u) = \mu \sum_{p \in P} k(u - p),$$

where k is a kernel that determines how the *offspring* points are distributed around the parents.

Note Neyman-Scott processes always have the parent and offspring intensities as parameters. Additional parameters are introduced by the choice of kernel.

Definition 3.3: Homogeneous Thomas process A homogeneous Thomas process is a Neyman-Scott process with an isotropic Gaussian kernel $\mathcal{N}(0, \sigma^2 I)$

$$\rho_{Thomas}(u) = \mu \sum_{p \in P} k(0, \sigma^s),$$

This process has parameters $\theta_{Thomas} = (\kappa, \mu, \sigma)$ corresponding to the parent intensity, offspring intensity, and cluster scale, respectively.

[TODO: add the Matérn cluster and nested Thomas definitions here – both are used in Section 5 but never defined]

3.2 Persistent homology

Radially averaged statistics – such as the g and K functions – collapse a point cloud to a real-valued curve indexed by scale r. Features based on such statistics are constructed by evaluating over a chosen range $[r_{min}, r_{max}]$ and subsampling at n points along the interval to obtain a feature vector in $\mathbb{R}^n$. This approach is limited because it discards multi-scale connectivity information and void structure, i.e. *how* the density is distributed across the space. Persistent homology retains this information by tracking a family of simplicial complexes on the point cloud as the scale parameter grows, instead of a single scalar summary. Under a filtration such as Vietoris-Rips, the H_0 persistence summary records the order and scales at which clusters merge. The H_1 counterpart marks voids and gaps in the pattern as they persist across scales, allowing one to capture the clustering-induced emptiness or repulsion that a process may exhibit, which a radial-average cannot see as clearly. For each $d \in \mathbb{N}$, the d-dimensional persistence summary requires no closed form expression and no assumption about which process family generated it. [TODO: Notation clash with 'd']

Let X be a point process on R^d and $W \subseteq \mathbb{R}^d$ a bounded window with $|X \cap W| = n$. Enumerate the points of $X \cap W$ to construct a point cloud $\mathbf{x} = \{x_1, ..., x_n\} \subset \mathbb{R}^d$. Let $\Sigma(\mathbf{x})$ denote the *full simplex* on $\mathbf{x}$, i.e. the collection of all non-empty subsets of $\mathbf{x}$

Definition 3.4: Simplex A *simplex* of $\mathbf{x}$ is a non-empty finite subset $\sigma \subseteq \mathbf{x}$. Its faces are the non-empty subsets $\tau \subseteq \sigma$.

Definition 3.5: Simplicial complex A simplicial complex Σ on $\mathbf{x}$ is a collection of simplices closed under taking faces: $\sigma \in \Sigma, \tau \subseteq \sigma \implies \tau \in \Sigma$.

Definition 3.6 A filtration function on $\mathbf{x}$ is a monotone map $f : \Sigma(\mathbf{x}) \to \mathbb{R}^+$ such that

$$f(\tau) \leq f(\sigma) \quad \forall \quad \tau \subseteq \sigma.$$

Example Vietoris-Rips filtration function:

$$f_{VR}(\{x_i\}) = 0, \qquad f_{VR}(\{x_i, x_j\}) = \|x_i - x_j\|,$$

extended to higher simplices by $f_{VR}(\sigma) = \max_{\{x_i, x_j\} \subseteq \sigma} f_{VR}(\{x_i, x_j\})$, i.e. the diameter of σ.

Example For $u \in \mathbb{R}^d$, let $N_k(u)$ denote its k nearest neighbours in $\mathbf{x}$. The DTM_k filtration function is

$$f_{DTM}(\sigma; k) = \max_{u \in \sigma} \{ (\frac{1}{k} \sum_{v \in N_k(u)} d(u, v)^2)^{\frac{1}{2}} \}$$

again extended to higher simplices by the max-over-edges rule of the VR example. [Anai et al., 2020].

Claim 3.7. *f_{DTM_k} is monotone, and is hence a valid filtration function [Anai et al., 2020, Prop. 3.5].*

Definition 3.8 Given a filtration function f on $\mathbf{x}$ and $r \geq 0$, we construct

$$\mathcal{F}_r = \{\sigma \in \Sigma(\mathbf{x}) : f(\sigma) \leq r\}.$$

Monotonicity of f ensures each $\mathcal{F}_r$ is a simplicial complex and that $\mathcal{F}_s \subseteq \mathcal{F}_t$ for $0 \leq s \leq t$. The family $(\mathcal{F}_r)_{r \geq 0}$ is the *filtration* induced by f. We write $VR_r(\mathbf{x})$ for $\mathcal{F}_r$ under f_{VR}.

Definition 3.9: Betti number Fix $k \in \mathbb{N}$ and a simplicial complex Σ. Taking homology with coefficients in a field (we use $\mathbb{Z}_2$), $H_k(\Sigma)$ is a vector space of dimension $\beta_k = \dim H_k(\Sigma)$, the k-th *Betti number*. $\beta_0, \beta_1, \beta_2, \ldots$ count connected components, loops, and cavities, respectively.

Definition 3.10: Persistence pair & lifetime For a filtration $(\mathcal{F}_r)_{r \geq 0}$, a k-dimensional homology class is *born* at the smallest scale $b \geq 0$ at which it appears in $H_k(\Sigma_b)$, and *dies* at the scale $d > b$ at which it merges into an older class or becomes a boundary. The pair (b, d) is a *persistence pair*, and $d - b$ its *lifetime*.

Example Consider H_0. Every 0-dimensional class is born at $r = 0$ and dies when its two components merge, i.e. at the filtration value of the connecting edge.

Definition 3.11: Persistence diagram A persistence diagram $D_k = \{(b_i, d_i)\}_i$ is the multiset of dimension-k persistence pairs, plotted with birth on the x-axis and death on the y-axis.

Remark Every point of D_k lies above the diagonal $b = d$ because it must die after it is born. Distance from the diagonal is lifetime, so short-lived points near the diagonal are often treated as noise.

Definition 3.12: Persistence image The persistence image of D_k is constructed by:

1. Mapping to birth-persistence coordinates: $(b, d) \mapsto (b, d - b)$.
2. Weighting each point by $w(d - b)$.
3. Forming the persistence surface $\rho(z) = \sum_i w(d_i - b_i)\, \phi(z; (b_i, d_i - b_i))$, a weighted sum of isotropic Gaussian kernels with bandwidth σ_{im} centered at the transformed points.
4. Integrating ρ over a 64×64 pixel grid, giving a single-channel image $PI \in \mathbb{R}^{64 \times 64}$.

Definition 3.13: Betti curve For fixed homology dimension k, the Betti curve is the right-continuous step function counting k-features alive at scale r:

$$\beta_k(r) = \#\{i : b_i \leq r < d_i\} = \sum_i \mathbb{1}[b_i \leq r < d_i].$$

[TODO: list here the OTHER vectorizations that were implemented and tested – they are the objects compared in Section 7.1 and must be defined somewhere]

3.3 Background reading

[TODO: WRITE LAST. Reading clusters to cover: (a) classical SPP estimation – Diggle, minimum contrast, Palm likelihood, composite likelihood; (b) TDA foundations – persistence, stability, DTM filtrations; (c) vectorization of persistence – persistence images, landscapes, Betti curves, kernel methods; (d) neural/simulation-based inference – amortized inference, neural estimators, the two related works of Section 2; (e) TDA applied to point processes and to cosmology.]

4 Problem Setup

Let X be a Neyman-Scott process on $\mathbb{R}^d$ with unknown parameter vector $\theta \in \mathbb{R}^{n_{params}}$. For example, the homogeneous Thomas process has $\theta = (\kappa, \mu, \sigma)$. We observe a single realization of X on a bounded window $W \subset \mathbb{R}^d$, giving a point cloud $\mathbf{x} = \{x_1, \ldots, x_n\} \subset \mathbb{R}^d$ with $n = |X \cap W|$. The

objective is to produce an estimate $\hat{\theta} = \hat{\theta}(\mathbf{x})$ of θ from $\mathbf{x}$ alone. We stress *single realization*: θ is not recovered from repeated draws of X, nor is $\mathbf{x}$ pooled or averaged across realizations. This is the setting a practitioner faces: one observed pattern and one chance to estimate the generating parameters. It is also what makes the problem hard in a way no estimator can fix: a fixed θ already induces a distribution over point clouds, so even the true parameter value gives rise to a spread of plausible $\mathbf{x}$, and it is this irreducible spread —- not any weakness of a particular estimator -— that sets the error floor characterized below.

The model is trained on a training set of synthetic clouds $\{\mathbf{x}_i, \theta_i\}_{i=1}^{N_{train}}$ by backpropagating the RMS loss

$$\mathcal{L}(\hat{\theta}_i, \theta_i) = \sqrt{\sum_{j=1}^{n_{params}} (\hat{\theta}_i^j - \theta_i^j)^2},$$

where $\theta_i = (\theta_i^1, \ldots, \theta_i^{n_{params}})$. Since parameters on a larger scale dominate this loss, we train in log-normalized parameter space,

$$\{\mathbf{x}_i, \theta_i\}_{i=1}^{N_{train}} \to \left\{\mathbf{x}_i, \frac{\log(\theta_i) - \mu_{\log\theta}}{\sigma_{\log\theta}}\right\}_{i=1}^{N_{train}},$$

where – abusing notation – $\mu_{\log\theta}, \sigma_{\log\theta}$ denote the mean and standard deviation of the logged parameters.

We draw one realization per parameter vector. The dispersion of $\hat{\theta}$ across realizations sharing a common θ gives the irreducible error floor against which the model's performance should be read.

[TODO: make the error floor a named quantity here and reference it from Section 7.3 – it is the yardstick for every reported number]

5 Simulator and Experimental Design

Training data consists of pairs $(\mathbf{x}_i, \theta_i)$ in which θ_i is drawn from a design distribution Π over parameters space and each $\mathbf{x}_i$ is a single realization of the corresponding process. We describe the design distribution, sampling algorithm, and verification procedure used to generate the synthetic data.

5.1 Design distribution

The naive approach of sampling (κ, μ, σ) uniformly and independently from intervals is inadequate. Firstly, point processes can typically be separated into qualitative regimes of interest, which occupy curved subsets of the parameter grid. Therefore uniform sampling would allocate most of its mass to configurations that are either near-Poisson or degenerate. Secondly, the raw parameters are not comparable across families, for example Matérn's ball radius R and Thomas's cluster scale σ do not play equivalent roles. We therefore sample in reparameterized coordinates that carry the same meaning for every family considered:

$\lambda = \kappa\mu$	expected offspring intensity,
κ	parent intensity,
$\nu = r\sqrt{\kappa}$	dimensionless overlap index,

where r denotes the cluster scale of the family in question: $r = \sigma$ for Thomas, $r = R$ for Matérn cluster, and $r = \sqrt{\sigma_1^2 + \sigma_2^2}$ for nested Thomas. The overlap index ν measures cluster radius against mean inter-parent spacing, so $\nu \ll 1$ gives well-separated clusters and $\nu \gtrsim 1$ gives clusters that merge (so the process approaches complete spatial randomness). Nested Thomas carries an additional coordinate, the scale ratio $\rho = \sigma_2/\sigma_1 \in (0, 1)$, which separates configurations in which both levels of structure are resolvable from those in which the process collapses to a single scale.

We draw $(\log \kappa, \log \lambda, \log \nu)$ uniformly over a grid and invert to recover $\mu = \lambda/\kappa$ and $r = \nu/\sqrt{\kappa}$. Log-uniform sampling is used because the parameters span several orders of magnitude, and it matches the log-normalization applied to the ground-truth targets (see Section 4) so that training targets are approximately uniform in each coordinate. Draws are taken from a scrambled Sobol sequence.

The ranges of the design grid are constrained as follows:

1. **Point budget.** The cost of computing persistence summaries grows rapidly with the number of points, so we impose $\mathbb{E}[N] = \lambda \in [150, 800]$, fixing the range of $\log \lambda$.

2. **Clusters per window.** We require $\kappa|W| = \kappa \geq 15$ so that a single realization is informative about κ, and $\kappa|W| = \kappa \leq 120$ so that individual clusters remain resolvable.

3. **Overlap.** We impose $\nu \in [0.10, 0.90]$. The upper limit deliberately admits the near-Poisson regime so that the model is still exposed to configurations in which the parameters are weakly identifiable.

Parameter draws violating these constraint are rejected and redrawn before any cloud is generated. This filter acts on the design and not on realizations, therefore it does not distort the conditional law of $\mathbf{x}$ given θ. The realized acceptance rate is 24.8%.

5.2 Sampling algorithm

The various instances of a the Neyman–Scott construction may be generated by a single algorithm. The only part that is different for each instance is the displacement kernel k and the offspring count distribution. For a window $W \subset \mathbb{R}^d$:

1. Expand W by an edge buffer b to produce W_{exp}, so that clusters whose parents lie outside W are not truncated.

2. Sample the parent process on W_{exp}: $n \sim \text{Poisson}(\kappa|W_{exp}|)$, with parents placed uniformly on W_{exp}.

3. For each parent p independently, draw an offspring count $n_p \sim \text{Poisson}(\mu)$.

4. For each offspring independently, draw a displacement $\epsilon \sim k$ and place the point at $p + \epsilon$.

5. Discard the parents and retain the offspring lying in W.

For the Thomas process $k = \mathcal{N}(0, \Sigma^2 I_d)$; for the Matérn cluster process k is uniform on the ball of radius R; for nested Thomas the parent process at step 2 is itself a Thomas process rather than a homogeneous Poisson process.

The buffer is set by a single rule applied to whichever kernel is in use: b is the radius containing all but ε of the mass of k. For a Gaussian kernel this gives $b = 4\sigma$, at which the per-axis tail mass is $2\Phi(-4) \approx 6.3 \times 10^{-5}$; for the compactly supported Matérn cluster kernel it gives $b = R$

exactly, with no tail margin required; for nested Thomas the buffers of the two levels add, giving $b = 4\sigma_1 + 4\sigma_2$, which over-covers the tight bound $4\sqrt{\sigma_1^2 + \sigma_2^2}$ and is therefore conservative. Points are never clipped to the boundary; the only boundary operation is the containment filter at step 5.

5.3 Validation

Throughout this section, let X denote a point process on $\mathbb{R}^d$. We write $u, v \in \mathbb{R}^d$ for points (precise locations) and du, dv for infinitesimal regions of volume $|du|$, $|dv|$ centered at u and v, respectively.

Definition 5.1: Second-order product density The second-order product density $\rho^{(2)} : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}^+$ is defined such that $\rho^{(2)}(u, v)\, du\, dv$ is the probability of jointly observing a point of X in each of the infinitesimal regions du and dv.

Definition 5.2: Pair correlation function The pair correlation function $g(u, v) : R^d \times R^d \to \mathbb{R}^+$ is defined

$$g(u, v) = \frac{\rho^{(2)}(u, v)}{\rho(u)\rho(v)}.$$

It is clear that for any Poisson process we have $g \equiv 1$. Intuitively, g can suggest whether SPPs are attractive ($g > 1$), or repulsive ($g < 1$).

Definition 5.3: Stationarity & isotropy A point process X is said to be:

- *Stationary* if $X \sim X + s$ for every $s \in \mathbb{R}^d$ where $X + s = \{x + s : x \in X\}$.
- *Isotropic* if $X \sim RX$ for every rotation R about the origin.

Note Stationarity implies constant intensity $\rho(u) = \lambda$, but the converse is not necessarily the case. If X is stationary and isotropic, $g(u, v)$ depends on u and v only through their inter-point distance $r = \|u - v\|$. Abusing notation, we may write $g(r)$ for this common value. We now define the K function.

Definition 5.4: K-function For stationary and isotropic X, the K-function $K(r) : \mathbb{R}^+ \to R^+$ is defined

$$K(r) = \int_{b_r(0)} g(\|h\|)dh$$

Intuitively, $\rho(u)K(r)$ is the expected number of points we expect to see within a distance r of a typical point in X.

[TODO: consider double duty for these definitions: they also give the cleanest statement of what a radially averaged second-order statistic CANNOT see, which is the argument Section 3.2 needs. Cross-reference rather than duplicate.]

Due to every downstream result being dependent on a diligent cloud-generation process, we validate it against the established closed-form second-order theory. On $W = [0, 1]^2$ we compare [[3000]] realizations at each of three parameter triples spanning the design, chosen to span the overlap index: a tight-cluster regime ($\nu = 0.07$), a moderate regime ($\nu = 0.22$), and a near-Poisson regime ($\nu = 1.00$). [TODO: fix the [[3000]] placeholder]

For the first moment, $\mathbb{E}[N(W)] = \kappa\mu|W|$. For the second, the canonical identity $\mathrm{Var}[N(W)] = \kappa\mu(1 + \mu)|W|$ holds only in the limit of a window large relative to σ, and is badly biased in precisely the near-Poisson regime we wish to stress. Applying the law of total variance to $N(W) =$

$\sum_p N_p(W)$ with $N_p(W) \mid p \sim \text{Poisson}(\mu q(p))$, $q(p) = \int_W k(x-p)\,dx$, and evaluating the shot-noise term by Campbell's formula gives the exact finite-window result

$$\text{Var}[N(W)] = \kappa\mu + \frac{\kappa\mu^2}{4\pi\sigma^2} J(\sigma)^2, \qquad J(\sigma) = \int_{-1}^{1} (1-|t|)\, e^{-t^2/4\sigma^2}\, dt, \tag{1}$$

for $W = [0,1]^2$, where the reduction to a one-dimensional integral follows from the triangular density of the difference of two independent Uniform$(0,1)$ variables. As $\sigma \to 0$, $J(\sigma) \to 2\sigma\sqrt{\pi}$ and (1) collapses to $\kappa\mu(1+\mu)$, confirming the two agree in the appropriate limit. The gap between them reaches 175% at $\nu = 1$, so (1) is used as the comparison target.

[TODO: FIGURE: canonical identity vs (1) as a function of ν, with the empirical points overlaid. This makes the 175% claim visible in one glance and is the cheapest high-value figure in the paper.]

Case	ν	$\mathbb{E}[N]$ theory	empirical	$\text{Var}[N]$ theory	empirical
A	0.07	200.0	200.4 ± 0.6	982.0	972.5 ± 25.1
B	0.22	100.0	100.2 ± 0.4	545.2	544.8 ± 14.1
C	1.00	20.0	20.0 ± 0.1	43.6	46.0 ± 1.2

Table 1: Simulator validation against (1). All deviations are within 2 standard errors.

We additionally verify that the empirical pair correlation function and Ripley's K match

$$g(r) = 1 + \frac{e^{-r^2/4\sigma^2}}{4\pi\kappa\sigma^2}, \qquad K(r) = \pi r^2 + \frac{1 - e^{-r^2/4\sigma^2}}{\kappa},$$

that the marginal offspring intensity on W shows no boundary deficit (edge-to-interior density ratio 1.003–1.048 across the three cases, in all cases consistent with unity and never in the direction an insufficient buffer would produce), and that the displacement kernel is isotropic with per-axis standard deviation matching σ. Estimates of g normalized per realization carry a negative bias that grows with $\mathbb{E}[N^2]$ and is therefore larger for clustered patterns than for a matched-intensity Poisson control; we account for this when reading residual deviations rather than attributing them to the sampler.

5.4 Splits and evaluation protocol

Data are split by parameter vector, never by realization, so that no parameter value appears in both training and evaluation. Beyond the random test split we generate two further evaluation sets: a grid over the design grid, permitting error to be reported as a surface over $(\log\nu, \log\lambda)$ rather than as a single scalar; and a set drawn one increment outside each range, characterizing degradation under extrapolation.

The classical baselines introduced in Section REF are run on the same clouds, under the same train/test/grid/extrapolation splits rather than on separate data or numbers taken from the literature. Section REF is therefore a comparison against a shared test set.

Throughout Section 7 we report error in the log-normalized parameter space of Section 4. For a parameter vector with n_{params} components and a test set of size N_{test}, we define the per-parameter loss

$$\mathcal{L}^j = \frac{1}{N_{test}} \sum_{i=1}^{N_{test}} (\hat{\tilde{\theta}}_i^j - \tilde{\theta}_i^j)^2, \qquad \mathcal{L} = \frac{1}{n_{params}} \sum_{j=1}^{p} \mathcal{L}^j,$$

where $\tilde{\theta}$ denotes the log-normalized parameter vector and $\hat{\tilde{\theta}}$ the model's estimate of it. Scalar figures quote $\mathcal{L}$; where a single number obscures the behaviour of an individual parameter we report the breakdown $\mathcal{L}^1, \ldots, \mathcal{L}^{n_{params}}$ alongside it.

Two remarks. Firstly, $\mathcal{L}$ is the training objective of Section 4 up to the averaging over parameters and the square root, so the quantity being optimized and the quantity being reported differ only in scale; we report $\mathcal{L}$ rather than the training loss because the per-parameter decomposition is not available from the latter. Secondly, because the standardization is the one used at training time, $\mathcal{L}^j$ is measured in units of the training-set variance of $\log \theta^j$. A value of $\mathcal{L}^j = 1$ therefore corresponds to an error of one standard deviation of $\log \theta^j$ over the design distribution, which is the error of a model that has learned nothing beyond the mean of the design distribution. This gives the upper reference point for every figure reported in Section 7; the lower reference point is the irreducible error floor of Section 4.

We additionally report error in the original parameter units where a physical reading is wanted, noting that these are not comparable across parameters and are not the quantity being optimized. [TODO: confirm the reported figures in Section 7.2 are $\mathcal{L}$ as defined here, and not the un-averaged training loss]

6 Method: ParamNet

Let $\mathbf{x}$ denote a point cloud, v its vectorized persistence summary, and $e \in \mathbb{R}^{128}$ the resulting feature vector. ParamNet follows the pipeline

$$\mathbf{x} \xrightarrow{\text{Vectorization}} v \xrightarrow{\text{Encoder}} e \xrightarrow{\text{FCNN}} \hat{\theta}. \tag{2}$$

We note that the pipeline shape follows the neural-estimator patterns used in REF and REF. Our contribution lies in the feature-extraction stage, the application to a wider range of SPP families, and the careful evaluation of design choices.

The model is trained and evaluated on synthetic clouds generated by sweeping a design distribution of the parameter space. The synthetic clouds are processed together before training so that the feature-extraction step is performed only once.

Feature extraction is the model's key component: it is where the cloud's topological summary is vectorized and made usable for machine learning. A substantial body of work has developed vectorizations of persistence diagrams —- persistence images, Betti curves, landscapes, kernel methods – that trade off stability, resolution, and information loss in different ways, but there is little consensus on which is preferable for recovering generating parameters from a single observed point cloud. We implement and compare several such vectorizations against this task directly, rather than picking one on prior grounds; the comparison, and what it reveals about the structure each vectorization does and does not preserve, is the subject of Section 7.1.

The vectorized summary v is passed through an encoder to obtain the feature vector $e \in \mathbb{R}^{128}$, which is then given to the FCNN $NN_\phi : \mathbb{R}^{128} \to \mathbb{R}^{n_params}$ to produce the estimate $\hat{\theta} = (\hat{\kappa}, \hat{\mu}, \hat{\sigma})$. NN_ϕ is a small feed-forward network: two hidden layers of width 64 and 32, each followed by a ReLU nonlinearity and dropout ($p = 0.1$), then a final linear layer mapping to the $n_{params} = 3$ outputs $(\hat{\kappa}, \hat{\mu}, \hat{\sigma})$.

6.1 Filtration and vectorization

We implemented and tested [TODO: N] vectorizations of the persistence diagram, whose relative efficacies are the subject of Section 7.1. For the remainder of this section we describe the pipeline

as instantiated with persistence images computed under the DTM_k filtration, which is the configuration carried through the rest of the paper. We give this configuration in full because it is the one that raises design questions the alternatives do not: the birth and persistence axes must be calibrated to a common frame before the diagrams can be rasterized, and computing summaries at several values of k leaves open how the resulting images are to be encoded and combined.

The pipeline is as follows;

1. Pick filtration function and construct filtrations $\{(\mathcal{F})_{r\geq 0}\}_i$.
2. Compute persistence diagrams $\{D^{(0)}\}_i$, $\{D^{(1)}\}_i$.
3. Calibrate diagrams to determine common birth/persistence axis ranges, shared across the dataset, then vectorize each diagram into a fixed-size raster using those ranges (see Definition 3.12).

The key design choices that follow are the filtration function, the calibration procedure, and the image resolution. The latter two can be optimized with a grid-search, but the choice of filtration function plays a fundamental role in what information is available to be extracted at all. We use the DTM_k filtration of Example 3.2 rather than the canonical Rips filtration: Rips is built from inter-point distances alone, so it is sensitive to the sparsest connection between two regions and does not register how densely those regions are occupied, whereas DTM_k averages over k nearest neighbours and so responds to local density directly. Since the parameters we wish to recover are density parameters, we expect this to matter, and Section 7.1 reports the comparison that bears this out. Furthermore, the parameter k allows us to compute summaries at various scales, which may be combined to provide the model with a more complete picture.

6.2 Calibration

Consider the synthetic training clouds $\{\mathbf{x}_i, \theta_i\}_{i=1}^{N_{train}}$ and fix the homology dimension $h \in \{0, 1\}$ for some choice of filtration. We obtain N_{train} persistence diagrams $\{D_i^{(h)}\}_{i=1}^{N_{train}}$. Each diagram will have a different distribution of points, some of which may have extreme outliers with respect to the full set of diagrams.

In order to ensure that in the resulting persistence images each pixel bears the same meaning, we need to calibrate the diagrams to have the same axis ranges. The naive approach of calibrating to the maximum deaths and births in order to include all points is inadequate due to the resulting relative positioning of the bulk of points.

If we have a diagram with very strong outliers and chose universal axes that include these outliers, the remaining diagrams will be rescaled in a way that places the majority of their points near the origin. The diagrams – and therefore their vectorizations – will lose a lot of texture.

[TODO: FIGURE: naive min/max calibration vs coverage-quantile calibration, same diagram, side by side. The "loss of texture" argument is visual and is currently carried entirely by prose.]

Our solution is to calibrate according to *per-diagram coverage quantiles*. For each training diagram $D_i^{(h)}$, define the three summary statistics

$$b_i^{\min} = \min_{(b,d)\in D_i^{(h)}} b, \qquad b_i^{\max} = \max_{(b,d)\in D_i^{(h)}} b, \qquad p_i^{\max} = \max_{(b,d)\in D_i^{(h)}} (d-b), \tag{3}$$

i.e. the smallest birth, largest birth, and largest persistence value attained *within a single diagram*. Given a coverage level $q \in (0, 1)$ and a padding factor $\alpha \geq 1$, we set the calibrated birth and

persistence ranges to be the q-quantiles of these per-diagram statistics, taken over the training set:

$$\left[\, Q_{1-q}\big(\{b_i^{\min}\}_{i=1}^{N_{train}}\big),\ \alpha \cdot Q_q\big(\{b_i^{\max}\}_{i=1}^{N_{train}}\big)\,\right] \qquad \text{and} \qquad \left[\, 0,\ \alpha \cdot Q_q\big(\{p_i^{\max}\}_{i=1}^{N_{train}}\big)\,\right], \tag{4}$$

where $Q_q(\cdot)$ denotes the empirical q-quantile. In practice we take $q = 0.99$ and $\alpha = 1.05$, which were determined via grid-search. These two ranges are shared by every diagram of homology dimension h, for every point cloud in the dataset (train, validation, test, and adversarial alike), so that a given pixel of the resulting persistence image always refers to the same region of birth-persistence space.

The key difference with the naive min/max calibration is *what* is being quantiled. Pooling all points from all diagrams and taking a quantile over that pooled set implicitly weights each diagram by how many topological features it produces: a single diagram with an unusually large number of points – near the diagonal or otherwise – can dominate the resulting bound, while a diagram consisting of one strong outlier and few other points contributes only that outlier. By instead computing one representative statistic per diagram and quantiling *over diagrams*, every training cloud contributes exactly once to the calibration, regardless of how many persistence pairs it happens to produce. Framing outlier rejection as a coverage fraction over diagrams – rather than over points – makes the trade-off explicit and controllable: at most $\lceil (1-q)N_{train} \rceil$ diagrams may have their most extreme birth or persistence value fall outside the calibrated frame (and are clipped rather than discarded), while the bulk of the population retains full resolution. The naive approach of calibrating to the global maximum is recovered as the degenerate case $q \to 1$.

This scheme is also compatible with the stability guarantees of persistence images [Adams et al., 2017], which are stated for a fixed bandwidth and a fixed image domain: since σ_{pixels} is specified in pixel units (Definition 3.12) rather than in absolute birth/persistence units, and the birth and persistence ranges are calibrated independently of one another, a pixel remains a comparable, well-defined location across every image built from the calibrated imager – irrespective of the very different physical spans the two axes typically have.

Because the calibrated ranges are a property of the training distribution, they must be fit *exclusively* on $\{D_i^{(h)}\}_{i \in \text{train}}$, then frozen and applied unchanged to validation, test, and adversarial diagrams. Fitting the ranges on the full, unsplit population of clouds – before the train/validation/test partition is drawn – lets information about the held-out clouds leak into the calibration used to build every training image, since every pixel's meaning is defined relative to the calibrated bounds. As the train/validation/test partition is redrawn per random seed, calibration must likewise be refit per seed, on that seed's training rows only, and then applied frozen to the corresponding validation, test, and adversarial rows.

For homology dimension H_0, birth values are often identically zero (or otherwise carry negligible spread) under the chosen filtration, in which case $b_i^{\min} \approx b_i^{\max}$ for all i and the calibrated birth range collapses to a point. We detect this case explicitly and fall back to a one-dimensional vectorization of the diagram rather than fabricate an axis, since a degenerate range would make an entire column of pixels convey no information.

6.3 Encoder and fusion

Many variations of encoders were tested for this model. We will focus on the variation that produced the best results, noting that the encoding step is an important aspect that will require further study. [TODO: REWRITE – same problem as the vectorization subsection. State the design space here; report the outcome in Section 7.2.] [FIX: Naturally, different vectorization techniques require

different encoder architectures to produce the embedding vector. Similarly to Section REF, we focus on the rich example of the multi-k DTM_k persistence image model and defer to Section REF for an exposition and comparison of the different vectorizations and encoders used.]

Consider our training clouds $\{\mathbf{x}_i, \theta_i\}_{i=1}^{N_{train}}$. We pick K scales for our DTM_k filtration, and compute persistence images for D homology dimensions with resolution $H \times W$. Therefore for batch size B, the objective is to pick a suitable encoder architecture for the following transformation:

$$(B, K, D, H, W) \xrightarrow{\text{Encoder}} (B, 128). \tag{5}$$

When it comes to using multiple images per homology dimension for each cloud, a central question is whether each vectorization should go through a shared-weight encoder, or be assigned its own independent encoder.

The shared encoder is designed as follows: each (D, H, W) persistence-image stack is first augmented with two coordinate channels – fixed x- and y-coordinate grids linearly spaced over $[-1, 1]$ – following the CoordConv construction [Liu et al., 2018], so that the convolutional stack has access to absolute pixel position rather than having to infer it.

This matters here specifically because the birth and persistence axes are calibrated (Section 6.2): a fixed pixel location has a fixed, shared meaning across the dataset, and CoordConv lets the network exploit that directly instead of relearning it.

The coordinate-augmented $(D + 2, H, W)$ tensor is then passed through a small convolutional stack: three 3×3 convolution blocks with channel widths $(32, 64, 128)$, each followed by a ReLU, and by 2×2 max-pooling with spatial dropout between blocks (but not after the last one); the resulting feature map is reduced by adaptive average pooling to a single spatial location and projected by a linear layer to the embedding dimension E.

To apply this one encoder to all K scales, the k-axis is folded into the batch dimension for a single forward pass – one call over $B \cdot K$ images rather than K separate calls – so the weights are tied across scales by construction rather than by any explicit sharing mechanism.

This produces a (B, K, E) tensor. The next question concerns how the per-k summary is prepared for NN_ϕ. The options explored include:

1. Flat concatenation of the K per-scale embeddings into a single $(B, K \cdot E)$ vector
2. A small 1-D convolutional fusion over the ordered k-axis (two Conv1d layers, letting adjacent scales mix), followed by average-pooling over k into a fixed-width (B, F) vector whose size does not depend on K
3. The same convolutional fusion, but flattening rather than pooling over k, giving a $(B, F \cdot K)$ vector that preserves which position came from which scale at the cost of a width that again grows with K

[TODO: end this subsection by forward-referencing Section 7.2 rather than announcing the winner here]

7 Experiments

[TODO: opening paragraph: protocol (Section 5), metric, seeds, hardware, and how the three subsections relate]

7.1 Comparison of topological summaries

[TODO: WRITE THIS SUBSECTION. Much of the underlying work is already done; it needs to be recovered from the exploratory experiments and run to a consistent protocol (same seeds, same splits, same design grid).]

[TODO: TABLE: filtration $\times$ vectorization, test loss mean $\pm$ sd over seeds]

[TODO: FIGURE: per-parameter error by summary type – expect σ and κ to behave very differently]

7.2 Architecture ablations

Experiments suggested that a shared encoder works just as well as the independent option: we ran a sweep crossing *encoder mode* (shared-weight vs. independent-per-k) against *fusion mode*, five seeds per combination, on the same $DTM_{5,10,15}$ nested-Thomas setup used throughout. Restricting to the simplest fusion strategy (flat concatenation), the shared-weight encoder matched the independent one to within seed noise:

encoder mode	fusion	test loss (mean $\pm$ sd)	failed seeds
shared	concat	0.200 ± 0.009	$0/5$
independent	concat	0.217 ± 0.028	$0/5$

[TODO: promote to a proper `table` environment with a caption and label; report the full six-configuration cross, not just the two concat rows]

with a ten-seed run of the shared-weight configuration (0.205 ± 0.012) confirming the same order of magnitude. Since tying weights across scales also cuts the parameter count roughly K-fold and removes one design axis, we adopt the shared encoder going forward.

We determined once again that the simplest option was the most effective, and we concatenate the per-k outputs. The full sweep, crossing both encoder mode and fusion strategy (six combinations, five seeds each), showed that the two convolutional-fusion variants are not simply worse on average but *unstable*: individual seeds collapse to a test loss of ~ 0.9–1.0, while other seeds of the very same configuration land right on the concatenation baseline (~ 0.20).

Only the independent-encoder, flatten-pooled combination converged cleanly on all five seeds, but even so it did not outperform plain concatenation (0.212 ± 0.005 vs. 0.200 ± 0.009). Flat concatenation was therefore preferred not only for its simplicity, but because it was the only strategy – together with its shared/independent-encoder counterpart – that trained reliably across seeds without any sign of the collapse observed for the convolutional-fusion alternatives.

[TODO: FIGURE: per-seed test loss scatter across the six configurations. The instability finding is the interesting part and a table of means hides it.]

7.3 Estimation performance

Comparison against classical estimators. [TODO: minimum contrast on K and on g, and Palm likelihood, run on the SAME clouds (Section 5). This is the headline comparison and the only one that can substantiate the limitations claimed in Section 1.]

Error surfaces over the design grid. [TODO: error as a surface over $(\log \nu, \log \lambda)$ using the grid evaluation set. Expect degradation as $\nu \to 1$ (weak identifiability) – if the model degrades more gracefully than minimum contrast there, that IS the result.]

Per-parameter breakdown. [TODO: κ, μ, σ separately. They are not equally hard and a single scalar loss hides this.]

Performance relative to the irreducible error floor. [TODO: report error as a ratio to the single-realization floor defined in Section 4, so the numbers mean something absolute.]

Extrapolation. [TODO: the out-of-range evaluation set – degradation one increment outside each design range.]

Cross-family results. [TODO: Thomas / Matérn cluster / nested Thomas. Whether one model trained across families beats per-family models is an interesting question in itself, and the common reparameterization of Section 5.1 is what makes it askable.]

8 Discussion and Limitations

The design distribution is a prior. Finally, we note that the design distribution acts as a prior, so the trained model approximates a posterior summary under it, and its ranges must therefore cover the intended application domain. [TODO: expand – this is the single most important limitation of any amortized estimator and deserves a full paragraph, not a trailing sentence]

Computational cost. [TODO: persistence computation scales badly in n; this is why the point budget of Section 5.1 is capped at 800. State the cost explicitly and compare against the cost of minimum contrast.]

Single realization. [TODO: the estimator sees one realization; classical methods often assume the same, but say so explicitly]

Homogeneity and stationarity. [TODO: everything here assumes homogeneous, stationary, isotropic processes]

9 Future Work

[TODO: immediate: complete Section 7.3. Give this a timeline – it is the nearest-term deliverable.]

[TODO: near term: the encoder question flagged in Section 6.3 as requiring further study – attention over scales, set-based encoders operating on diagrams directly rather than on rasters]

[TODO: medium term: inhomogeneous / non-stationary processes, where classical closed forms are least available and the topological approach has the most to gain – this is arguably where the thesis-scale contribution lies]

[TODO: medium term: real data. Galaxy catalogues if the running example of Section 1 is cosmological; name a specific dataset.]

[TODO: longer term: uncertainty quantification – the model currently emits a point estimate with no interval, which limits its use as a drop-in replacement for likelihood-based methods]

[TODO: set up references.bib – currently only `anai2020dtm`, `adams2017persistence`, and `liu2018coordconv` are cited, and the Diggle / Tanaka references in Section 1 are inline text rather than citations]

References

Henry Adams, Tegan Emerson, Michael Kirby, Rachel Neville, Chris Peterson, Patrick Shipman, Sofya Chepushtanova, Eric Hanson, Francis Motta, and Lori Ziegelmeier. Persistence images: A stable vector representation of persistent homology. *Journal of Machine Learning Research*, 18 (8):1–35, 2017.

Hirokazu Anai, Frédéric Chazal, Marc Glisse, Yuichi Ike, Hiroya Inakoshi, Raphaël Tinarrage, and Yuhei Umeda. DTM-based filtrations. In *Topological Data Analysis: The Abel Symposium 2018*, pages 33–66. Springer, 2020. [TODO: verify page range and editor list].

Rosanne Liu, Joel Lehman, Piero Molino, Felipe Petroski Such, Eric Frank, Alex Sergeev, and Jason Yosinski. An intriguing failing of convolutional neural networks and the CoordConv solution. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.